

Ciclos en JavaScript:

Estructuras

- For
- while
- do-while.

Adecuación de extensiones:

Live Server (Ritwick Dey):

Lanza un servidor de desarrollo local con recarga en vivo.

Prettier - Code formatter:

Estandariza la indentación de los bloques de código. En las estructuras cíclicas, una mala indentación hace que el código sea ilegible y oculta visualmente los errores de lógica.

GitLens — Git supercharged:

Vital para la implementación de GitFlow. Permite visualizar directamente en el editor quién modificó cada línea de código (blame), facilita la revisión de ramas y simplifica la resolución de conflictos antes de un Pull Request.

ESLint:

Analiza el código estáticamente para encontrar problemas rápidamente. Alertará a los estudiantes si declaran variables en un bucle que nunca se utilizan o si hay riesgo de coerción de tipos.

Puente Cognitivo

Para esta sesión es imperativo dominar la **evaluación de expresiones booleanas (Condicionales)** y la **coerción de tipos**. Las estructuras cíclicas dependen de una condición (true o false) para saber si deben continuar ejecutándose o detenerse. Si la condición se evalúa incorrectamente, la aplicación fallará críticamente.

Reto: Analicen el siguiente código secuencial. Su tarea es hacer de "motor de JavaScript", leer el código de arriba hacia abajo y determinar exactamente qué mensajes aparecerán en la consola y cuál será el valor final de la variable batería.

```
let bateria = 100;
const consumoPorUso = 30;
if (bateria >= consumoPorUso) {
  bateria = bateria - consumoPorUso;
  console.log("Uso 1. Batería restante: " + bateria);
}
if (bateria >= consumoPorUso) {
  bateria = bateria - consumoPorUso;
  console.log("Uso 2. Batería restante: " + bateria);
}
if (bateria >= consumoPorUso) {
  bateria = bateria - consumoPorUso;
  console.log("Uso 3. Batería restante: " + bateria);
}
if (bateria >= consumoPorUso) {
  bateria = bateria - consumoPorUso;
  console.log("Uso 4. Batería restante: " + bateria);
}
```

Situación

En la sesión anterior, aprendimos a usar `if` para tomar una decisión. Por ejemplo, podríamos comprobar si la nota de *un* estudiante es mayor a 3.0 para saber si aprobó.

Ahora, ¿qué pasa si tenemos un salón con 40 estudiantes? ¿Escribimos 40 bloques `if` diferentes?

```
if (estudiante1.nota > 3.0) { ... } if (estudiante2.nota > 3.0) { ... } ... if (estudiante40.nota > 3.0) { ... }
```

Eso es ineficiente, imposible de mantener y viola el principio de "No Te Repitas" (DRY - Don't Repeat Yourself).

Necesitamos una forma de decirle a JavaScript:

Toma esta lista de estudiantes y **repite** esta misma lógica de `if` para **cada uno** de ellos.

Esa instrucción de "repetir" es un **ciclo** o **bucle**. Los bucles son el motor de nuestras aplicaciones; son la maquinaria que nos permite procesar miles de datos con unas pocas líneas de código.

El Concepto de Iteración

Un ciclo (o bucle) es una estructura de control que repite un bloque de código mientras una condición sea **verdadera (true)**. Todos los ciclos comparten tres componentes fundamentales:

1. **Inicialización:** El punto de partida (una variable contador).
2. **Condición:** La regla booleana que, mientras sea true, mantiene el ciclo vivo.
3. **Actualización (Incremento/Decremento):** La modificación de la variable inicial para asegurar que la condición en algún momento sea false y el ciclo termine.

El Ciclo for (La Iteración Definida)

El ciclo for es la estructura de repetición más común. Se usa cuando **sabemos de antemano cuántas veces** queremos que se repita una acción. (Ej. "Repita esto 10 veces", "Repita esto por cada uno de los 40 estudiantes").

Un ciclo for es como un entrenador que te dice: "Corre 5 vueltas a la pista".

1. **Punto de Partida:** Empiezas en la vuelta 0 (o 1).
2. **Condición para Seguir:** ¿Has corrido *menos de* 5 vueltas? Si es sí, sigue corriendo.
3. **Acción por Vuelta:** Al final de cada vuelta, anotas 1 vuelta más en tu contador.

Anatomía del Código

El for se define con tres partes clave dentro de los paréntesis, separadas por punto y coma: for (inicialización; condición; incremento) { ... }

// for (1. Inicialización; 2. Condición; 3. Actualización)

```
for (let i = 1; i <= 3; i++) {  
  console.log("Ejecución forzada número: " + i);  
}
```

- **1. let i = 0; (La Inicialización):** Es el "Punto de Partida". Creamos una variable (j usando let!) que actuará como nuestro contador. Esto solo se ejecuta **una vez** al inicio.
- **2. i < 5; (La Condición):** Es la "Regla para Continuar". Antes de cada vuelta, JavaScript revisa esta condición. Si es true, ejecuta el bloque de código. Si es false, el ciclo se detiene.
- **3. i++ (El Incremento):** Es el "Paso". Se ejecuta **después** de cada vuelta. i++ (de la Sesión 1) suma 1 a nuestro contador, acercándonos al final.

Profundización

¿Por qué let y no var? Si usáramos var i, esa variable i se "fugaría" del ciclo y existiría en todo nuestro código, pudiendo causar errores. Al usar let i, la variable i **solo existe dentro** del bloque {...} del ciclo, lo cual es mucho más seguro y predecible.

El Ciclo while

Usamos un while cuando **no sabemos cuántas veces** necesitamos repetir. La repetición no depende de un contador, sino de que una condición externa siga siendo verdadera.

Mientras el jugador tenga vidas ($\text{vidas} > 0$), sigue jugando. No sabes si el juego durará 1 minuto o 3 horas. El ciclo continúa mientras la condición se cumpla.

El Bucle Infinito:

El while es poderoso pero peligroso.

Si la condición nunca se vuelve false, el código se repetirá para siempre, congelando el navegador. **Algo dentro del while debe, eventualmente, cambiar la condición.**

Anatomía del Código

```
let vidas = 3;
```

```
while (vidas > 0) {  
  console.log(`¡Tienes ${vidas} vidas!`);
```

```
  // Simulamos que el jugador pierde una vida  
  // ESTA LÍNEA ES CRUCIAL. Modifica la condición.  
  vidas--; // Sin esto, sería un bucle infinito.  
}
```

```
console.log('Game Over');
```

El Ciclo do...while

Es casi idéntico al while, con una sola diferencia crucial: el do...while **siempre se ejecuta al menos una vez**.

Caso de uso:

Un menú de opciones donde primero muestras el menú y luego preguntas si quiere pedir algo del menú.

Se usa en situaciones donde necesitas que el código se ejecute una vez antes de comprobar si debe repetirse.

Anatomía del Código

```
let contrasena;
```

```
do {
```

```
  // Este bloque se ejecuta SIEMPRE al menos una vez
```

```
  contrasena = prompt('Introduce tu contraseña:');
```

```
} while (contrasena !== '12345'); // La condición se revisa DESPUÉS
```

```
console.log('¡Bienvenido!');
```

El Bucle for...of

Este es el sucesor moderno del ciclo for clásico cuando quieres recorrer un **Arreglo**. Es mucho más limpio y menos propenso a errores.

Para qué sirve:

Para iterar sobre los **valores** de un objeto "iterable" (como un Arreglo, un String, un Mapa, etc.).

Casos de Uso

- **Ciclo for (clásico):** Eres un repartidor al que le dicen: "Ve a la casa #0, luego a la casa #1, luego a la #2...". Te importa mucho el **índice** (el número de la casa).
- **Ciclo for...of:** Eres un repartidor al que le dan una lista de clientes. Vas al "cliente A", luego al "cliente B". No te importa si son la casa #0 o la #1(indice), solo te importa el **valor** (el cliente).

Anatomía del Código:

```
const miArray = ['manzana', 'banana', 'naranja'];

// El 'for...of'
for (const fruta of miArray) {
  // 'fruta' toma el VALOR de cada elemento en cada vuelta
  console.log(fruta);
}

// Resultado:
// "manzana"
// "banana"
// "naranja"
```

Cuando usarlo: Es la forma **preferida y moderna** de recorrer un arreglo cuando solo te importa el valor de cada elemento y no necesitas saber su posición (el índice i).

El Bucle for...in (El Iterador de Propiedades de Objetos)

Este bucle se ve similar, pero es **fundamentalmente diferente** y es una fuente común de confusión.

Para qué sirve:

Para iterar sobre las **propiedades** (las "claves" o *keys*) de un **Objeto**.

A este bucle no le importa quién vive en la casa (valor), le importa el nombre de las habitaciones (clave). Si le das un objeto persona, te dirá que tiene una habitación "nombre", una "edad" y una "profesión".

Nunca uses for...in para recorrer un **Arreglo**. Técnicamente "funciona", pero lo hará de forma incorrecta:

1. Iterará sobre los **índices** ('0', '1', '2') como si fueran strings, no sobre los valores.
2. Puede iterar sobre propiedades inesperadas que no son parte de los datos.

Resumen de la Diferencia:

- for...of -> Para **valores** de **Arreglos**.
- for...in -> Para **claves** de **Objetos**.

Anatomía del Código:

```
const persona = {  
  nombre: 'Ana',  
  edad: 30,  
  profesion: 'Doctora'  
};
```

```
// El 'for...in'  
for (const clave in persona) {  
  // 'clave' toma el NOMBRE de la propiedad en cada vuelta  
  console.log(clave); // 'nombre', 'edad', 'profesion'
```

```
// Para ver el valor, debes usar la clave  
  console.log(persona[clave]); // 'Ana', 30, 'Doctora'  
}
```

Mención Honorífica:

.forEach() (El Método de Array)

No es un "bucle" en el sentido de una sentencia de JavaScript, sino un **método** que tienen los arreglos. Es la forma "funcional" de hacer un for...of.

```
const miArray = ['manzana', 'banana', 'naranja'];
```

```
// El .forEach() recibe una función callback  
miArray.forEach(function(fruta) {  
  console.log(fruta);  
});
```

Control de Ciclos (break y continue)

A veces necesitamos alterar el flujo normal de un ciclo.

- **break (La Salida de Emergencia):** Termina el ciclo **inmediatamente**, sin importar si la condición se sigue cumpliendo.
 - **Uso:** Estoy buscando un nombre en una lista de 1000. Lo encontré en la posición 50. ¡No necesito revisar los 950 restantes! ¡break!
- **continue (Saltar esta Vuelta):** Detiene la iteración **actual** y salta directamente a la siguiente (al i++ en un for).
 - **Uso:** Estoy procesando 100 emails. El email actual es spam. No lo proceso (continue) y salto al siguiente email.

Interacción con el Usuario: prompt y alert

Para simular el dinamismo sin construir aún interfaces HTML complejas, interactuaremos directamente con las APIs del navegador (Window):

- **prompt("mensaje"):** Pausa la ejecución del *Call Stack* y despliega una caja de texto. Devuelve un String con lo escrito por el usuario, o null si presiona Cancelar. **Siempre debe parsearse a número** (Number() o parseInt()) si se usarán matemáticas.
- **alert("mensaje"):** Pausa la ejecución y muestra un cuadro de diálogo con un botón de "Aceptar".

Preguntas

¿Cuál es la diferencia real entre for y while?

Es la pregunta más importante. Usa for cuando tengas una **iteración definida** (sabes cuántas veces, ej: 10 vueltas, o una vez por cada elemento de una lista). Usa while cuando tengas una **iteración indefinida** (no sabes cuántas veces, ej: "hasta que el usuario escriba 'salir'", "mientras el jugador tenga vidas").

Mi navegador se congeló cuando usé un while. ¿Qué pasó?

Creaste un **bucle infinito**. Esto sucede cuando la condición de tu while empieza siendo true y nada dentro del bucle hace que se vuelva false. Por ejemplo: `let i = 0; while (i < 10) { console.log(i); }`. La variable i nunca cambia, siempre es 0, por lo que `0 < 10` siempre es true. La solución es añadir un incremento (`i++`) dentro del bucle.

¿Por qué siempre se usa la variable i?

Es una convención histórica que viene de "Índice" o "Iterador". Si tienes ciclos anidados (uno dentro de otro), se suele usar j, luego k. Sin embargo, puedes llamarla como quieras (contador, numero, etc.), pero i es el estándar universal.

Ejemplo: La Tabla de Multiplicar (for)

Vamos a imprimir la tabla de multiplicar del 7

```
const numeroBase = 7;
console.log(`--- Tabla del ${numeroBase} ---`);

for (let i = 1; i <= 10; i++) {
  let resultado = numeroBase * i;
  console.log(`${numeroBase} x ${i} = ${resultado}`);
}

// Salida:
// 7 x 1 = 7
// 7 x 2 = 14
// ...
```

Simularemos el desarrollo de un cajero de supermercado. El sistema debe solicitar precios iterativamente hasta que el cajero indique que no hay más productos (ingresando "0"), y al final mostrar el total.

```
"use strict";

let totalCompra = 0;
let precioProducto;

alert("Bienvenido al sistema de caja registradora.");

// Usamos do-while porque necesitamos pedir el precio AL MENOS una vez.
do {
  // 1. Capturamos el dato (Llega como String)
  let entrada = prompt("Ingrese el precio del producto (o '0' para finalizar y pagar:");

  // 2. Coerción explícita: Convertimos el String a Number
  precioProducto = Number(entrada);

  // 3. Validación de seguridad (Si no ingresa un número, isNaN devuelve true)
  if (isNaN(precioProducto) || precioProducto < 0) {
    alert("Error: Por favor ingrese un monto numérico válido y positivo.");
  } else if (precioProducto > 0) {
    // 4. Operación aritmética: Acumulamos el valor
    totalCompra = totalCompra + precioProducto; // También válido: totalCompra += precioProducto;
    console.log("Producto agregado. Subtotal actual: $" + totalCompra);
  }

} while (precioProducto !== 0); // La condición de salida es que el cajero ingrese el número 0

alert("El total a cobrar es: $" + totalCompra);
console.log("Transacción finalizada.");
```

Práctica acompañada

Vamos a construir el módulo de bloqueo de seguridad para una cuenta bancaria (Branch: feature/seguridad-login). El usuario tiene exactamente 3 intentos para ingresar la contraseña correcta ("admin123"). Si acierta, ingresa. Si falla 3 veces, el sistema se bloquea.

```
// Paso 1: Definir constantes para la configuración
const CONTRASENA_CORRECTA = "admin123";
const INTENTOS_MAXIMOS = 3;
let accesoConcedido = false;

// Paso 2: Usamos un for porque sabemos exactamente el límite de intentos (3)
for (let intento = 1; intento <= INTENTOS_MAXIMOS; intento++) {

    let passIngresada = prompt("Intento " + intento + " de " + INTENTOS_MAXIMOS + ". Ingrese su contraseña:");

    // Paso 3: Validar la entrada
    if (passIngresada === CONTRASENA_CORRECTA) {
        accesoConcedido = true;
        alert("Contraseña correcta. Bienvenido al sistema.");
        break; // Rompe el ciclo inmediatamente, no sigue iterando.
    } else {
        alert("Contraseña incorrecta.");
    }
}

// Paso 4: Evaluar el estado final al salir del ciclo
if (accesoConcedido === false) {
    alert("Has agotado tus intentos. Cuenta bloqueada por seguridad.");
}
```

Actividad de clase

Pair Programming:

- **Driver (Piloto):** Gestiona el editor y las herramientas de desarrollo.
- **Navigator (Copiloto):** Dicta la estructura lógica basándose en los requerimientos.

El Reto: Simulador de Menú Interactivo

1. Creen un menú usando un ciclo do-while.
 2. Dentro del ciclo, utilicen prompt para mostrar estas opciones y capturar la respuesta: "Elija una opción:\n 1. Consultar Saldo\n 2. Retirar Dinero\n 3. Salir"
 3. Utilicen un bloque switch (visto en la sesión anterior) para evaluar la respuesta del usuario.
 4. Si elige 1, usen alert("Su saldo es \$1000"). Si elige 2, usen alert("Dinero retirado").
 5. El ciclo debe repetirse indefinidamente **mientras** la opción ingresada sea diferente de "3".
 6. Si elige 3, el ciclo termina y un alert fuera del ciclo debe decir "Gracias por usar el sistema".
- Tiempo: 25 minutos. Intercambio de roles a la mitad.*

Práctica Integradora

Ticket JIRA-108: Generador de Tablas de Multiplicar Dinámicas

A. Instrucciones: El módulo escolar requiere que el usuario ingrese un número del 1 al 10 mediante prompt. El sistema debe calcular e imprimir en la consola la tabla de multiplicar de ese número (del 1 al 10) usando un ciclo for.

B. Código Base:

```
let numeroUsuario = prompt("Ingrese el número de la tabla que desea calcular:");
numeroUsuario = Number(numeroUsuario);
// Implementa el ciclo for aquí
```

C. Solución Completa:

```
let numeroUsuario = prompt("Ingrese el número de la tabla que desea calcular:");
numeroUsuario = Number(numeroUsuario);

if (numeroUsuario > 0 && numeroUsuario <= 10) {
  console.log("--- Tabla del " + numeroUsuario + " ---");
  for (let multiplicador = 1; multiplicador <= 10; multiplicador++) {
    let resultado = numeroUsuario * multiplicador;
    console.log(numeroUsuario + " x " + multiplicador + " = " + resultado);
  }
} else {
  alert("Número fuera de rango. Debe ser entre 1 y 10.");
}
```

Ticket JIRA-109: Validador de Datos Críticos

A. Instrucciones: Para el registro tributario, el usuario debe ingresar obligatoriamente un número de identificación válido (mayor a 0). Utiliza un ciclo while para seguir mostrando un prompt exigiendo el dato **hasta que** el usuario ingrese un valor mayor a 0 y que no sea NaN.

B. Código Base:

```
let idTributario = 0;  
// Implementa el ciclo while de validación aquí
```

C. Solución Completa:

```
let idTributario = 0;  
  
// Mientras el ID sea menor o igual a 0, o sea un NaN, repetimos.  
while (idTributario <= 0 || isNaN(idTributario)) {  
  let entrada = prompt("Ingrese un ID Tributario válido (número mayor a cero:");  
  idTributario = Number(entrada);  
}  
  
alert("Registro exitoso. Su ID es: " + idTributario);
```

Ticket JIRA-110: Juego de Estimación (Minijuego Backend)

A. Instrucciones: Crea un número secreto quemado en el código (ej. 7). Usa un ciclo do-while para pedir al usuario que adivine el número del 1 al 10. Si falla, dile con un alert si el número secreto es mayor o menor al ingresado. Repite hasta que acierte.

B. Código Base:

```
const NUMERO_SECRETO = 7;  
let intentoUsuario;  
// Implementa la lógica iterativa y condicional aquí
```

C. Solución Completa:

```
const NUMERO_SECRETO = 7;  
let intentoUsuario;  
  
do {  
  intentoUsuario = Number(prompt("Adivina el número secreto (1 al 10):"));  
  
  if (intentoUsuario < NUMERO_SECRETO) {  
    alert("El número secreto es MAYOR. Intenta de nuevo.");  
  } else if (intentoUsuario > NUMERO_SECRETO) {  
    alert("El número secreto es MENOR. Intenta de nuevo.");  
  }  
  
} while (intentoUsuario !== NUMERO_SECRETO);  
  
alert("¡Correcto! El número secreto era " + NUMERO_SECRETO);
```

Ejercicio Independiente Aplicado

Ticket JIRA-208 - Calculadora de Promedio Estudiantil:

El cliente requiere un script para profesores. El requerimiento dicta lo siguiente:

1. Solicitar al profesor (vía prompt) la cantidad total de estudiantes que tiene en clase. Convertir ese valor a Number.
2. Utilizar un ciclo for que se ejecute tantas veces como estudiantes haya indicado el profesor.
3. En cada iteración del ciclo, usar un prompt que pida: "Ingrese la calificación final del estudiante [número de estudiante actual]".
4. Acumular las calificaciones en una variable totalizadora. (Cuidado con la coerción de strings).
5. Al finalizar el ciclo, calcular el promedio (suma de calificaciones dividida por la cantidad de estudiantes).
6. Mostrar el promedio del grupo utilizando un alert.

Criterios de logro esperados:

- Validación inicial: Si el profesor ingresa "0" estudiantes o letras, el script debe abortar con un mensaje de error sin intentar ejecutar el ciclo.
- Uso correcto de la estructura for con el límite basado en la variable dinámica entregada por el profesor.
- Coerción correcta utilizando Number() en cada iteración de la captura de notas.
- Código correctamente indentado e insertado en la rama feature/calculadora-promedio.